

文章编号: 1001-9081(2019)S2-0132-04

通过重组超级块改善固态硬盘寿命和性能的算法

段星辉¹, 陈 仁^{2,3*}

(1. 黔南民族师范学院 计算机与信息学院, 贵州 都匀 558000;

2. 中国科学院 红外探测与成像技术重点实验室, 上海 200083; 3. 中国科学院 上海技术物理研究所, 上海 200083)

(* 通信作者电子邮箱 danny.duan@foxmail.com)

摘 要:针对预留空间较小的固态硬盘(SSD)在做垃圾回收(GC)时会引起严重的写放大(WA)的问题,以及写放大导致系统寿命折损和满盘下写性能差的问题,提出一种超级块重组算法,即在垃圾回收挑选源超级块的时候,通过挑选每个闪存 die 上有效数据量最小的物理块重组超级块,重组出来的超级块具有最小的有效数据量,然后把它作为垃圾回收的源超级块。实验结果表明,和传统的基于超级块的垃圾回收算法相比,超级块重组算法能减小 2/3 的写放大,系统寿命提升将近 3 倍,满盘下写入性能提升将近 5 倍。该算法能明显减少系统的写放大,对预留空间小但对系统寿命和满盘下写性能有要求的固态硬盘设计有很好的参考和实用价值。

关键词:固态硬盘; 闪存转换层; 垃圾回收; 写放大; 闪存

中图分类号: TP333.35 **文献标志码:** A

Algorithm of improving solid-state drive endurance and performance by reorganizing superblocks

DUAN Xinghui¹, CHEN Ren^{2,3*}

(1. School of Computer and Information, Qiannan Normal College for Nationalities, Duyun Guizhou 558000, China;

2. Key Laboratory of Infrared System Detection and Imaging Technology, Chinese Academy of Sciences, Shanghai 200083, China;

3. Shanghai Institute of Technical Physics, Chinese Academy of Sciences, Shanghai 200083, China)

Abstract: Garbage Collection (GC) results in Write Amplification (WA), which impacts the endurance and full drive write performance (including sequential write and random write) of the Solid-State Drive (SSD), especially for the SSD with low Over Provisioning (OP). To solve these problems, this paper proposed a superblock reorganization algorithm, that is, when the source superblock was selected by garbage collection, the superblock was reorganized by selecting the physical block with the smallest amount of valid data on each flash memory die. The reorganized superblock had the smallest amount of valid data and was then used as the source superblock for garbage collection. Compared with the traditional garbage collection algorithm based on superblock, the proposed superblock reorganization algorithm can greatly reduce the write amplification of the system. The experimental results show that the superblock reorganization algorithm can reduce the write amplification by two thirds, improve the system life nearly three times, and improve the full drive write performance nearly five times. This algorithm has a good reference and practical value for the design of solid-state drive with low OP but high requirement for system life and full drive write performance.

Key words: Solid-State Drive (SSD); flash translation layer; garbage collection; Write Amplification (WA); NAND flash

0 引言

近年来,由于固态硬盘(Solid-State Drive, SSD)具有速度快、延时低、功耗低、抗震、体积小等优点^[1],无论是在消费领域,还是在企业数据中心,固态硬盘都得到广泛的应用,并在这些领域慢慢取代传统磁硬盘。

固态硬盘的存储介质一般是闪存(NAND Flash)。闪存有其自身的一些特点,如写前擦除、擦除/写入次数有限、读取次数有限等^[2]。

写前擦除特性不允许数据覆盖更新,当一个逻辑页面被重写时,设备会分配一个新的可用闪存页,将新的数据写入其中,并通过闪存转换层(Flash Translation Layer, FTL)软件层

来维护逻辑页到闪存页的映射关系。同时,这种异地更新(out-of-place update)的数据写入方式,会导致闪存块上存在很多垃圾(无效或者过时的数据),因此,SSD内部都需要垃圾回收算法,即把源闪存块上的有效数据集中写到新的闪存块,从而释放出闪存空间。

闪存的另一个特性是擦写次数有限,即它是有一定寿命的。闪存从早期单层存储单元(Single Level Cell, SLC)的上万甚至几十万的擦写次数,到现在先进制程的三层存储单元(Triple Level Cell, TLC)的几千次擦写次数。由于每个物理块擦写次数有限,因此SSD需要实现磨损均衡算法(Wear Leveling),以避免有些闪存块写入次数过多而早早变成坏块。

除了擦写次数受限,闪存一个物理块上的读次数也是有

收稿日期:2019-05-30;修回日期:2019-06-29。 基金项目:上海市科学技术委员会科研计划项目(16XD1404100)。

作者简介:段星辉(1981—),男,江西抚州人,副教授,博士,主要研究方向:固态存储、大数据存储和管理; 陈仁(1980—),男,安徽马鞍山人,副研究员,博士,主要研究方向:星载光电仪器设计、傅里叶光谱仪。

限的,一个闪存块上过多的读操作会导致数据丢失,这就是读干扰(Read Disturb)。因此 SSD 内部需要读干扰处理算法来避免过多的读导致数据丢失。

在 SSD 内部,通过 FTL 实现了这些算法。除上面说到的闪存特性,闪存还有其他的很多特性,FTL 都需要实现相应的算法来解决这些问题。

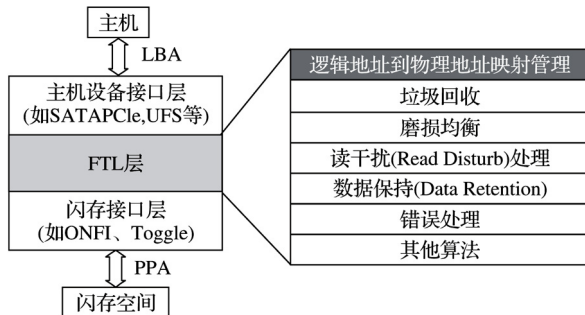


图1 FTL在固态硬盘设备中的位置和作用

在存储设备内部,FTL 首先起着承上启下的作用: 处理来自主机的读写请求,然后把这些读写请求转换成对闪存的访问请求; 另外 FTL 实现了一些和闪存特性相关的算法,诸如垃圾回收、磨损均衡、读干扰处理、数据保持处理等算法^[3], 来克服闪存的一些特性或者说是缺点。FTL 算法的好坏,直接影响着固态硬盘设备的性能、可靠性等关键指标。

1 基于超级块的垃圾回收

FTL 一般以超级块(SuperBlock)^[4]来管理闪存空间。超级块是所有闪存 die 具有同样编号的物理块(Block)的集合。如图2所示,超级块3是所有 die 上 block 3 的一个集合。FTL 分配闪存块、垃圾回收、读干扰处理等,都是以超级块为基本的单元。

	Die 0		Die 1		Die 2		Die 3	
	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1
Block 0								
Block 1								
Block 2								
Block 3	超级块(Super Block) 3							
Block 4								
Block 5								
Block 6								
Block 7								
Block 8								
⋮								

图2 超级块的概念

这种管理方式的最大好处是读写具有最大的并行度,因为读写超级块的时候,所有 die 上的物理块可以并行操作,从而性能最优。

FTL 记录了每个超级块上面的有效数据个数(Valid Page Count, VPC), 垃圾回收时,一般采用贪心算法^[5](Greedy Algorithm),即有效数据最小的那个超级块会被选为源超级块,然后有效数据会从该超级块读出,集中写到一个目标超级块。

如图3所示,由于超级块5的VPC最少(266),因而在下次垃圾回收时,根据贪心算法,它会被选为垃圾回收的源超级块。

垃圾回收的目标是垃圾回收释放出来的空间和主机写入的数据保持一个平衡,因此 FTL 在回收一个超级块时,主机

写入的数据量应该和该超级块上的无效数据相等,因此垃圾回收导致的写放大(Write Amplification, WA)^[6]为:

$$WA = TPC / (TPC - VPC) = 1 / (1 - VPC/TPC) \quad (1)$$

其中: WA 定义为写入到闪存的数据量和主机写入的数据的比; TPC(Total Page Count) 指一个超级块总共能写入的数据量, VPC 指源超级块上面有效的数据量, TPC - VPC 则是该超级块上面的无效数据个数。

写放大越大,意味同样的主机数据写入量,写到闪存上的数据量越多,对闪存的磨损(寿命)和性能影响越大,因此 SSD 设计的一个重要目标是减小写放大。从式(1)可以看出,源数据块上面有效数据越少,则写放大越小,从而对闪存的寿命和性能影响越小,反之亦然。

	Die 0		Die 1		Die 2		Die 3		有效数据个数
	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	
Block 0	40	41	99	97	42	12	4	52	387
Block 1	49	23	93	4	48	85	0	83	385
Block 2	40	56	98	42	52	7	47	53	395
Block 3	95	50	65	18	68	2	12	11	321
Block 4	29	49	33	7	4	52	68	72	314
Block 5	75	10	45	14	57	24	30	11	266
Block 6	17	57	64	88	88	50	69	39	472
Block 7	98	73	91	95	15	90	71	39	572
Block 8	67	58	82	36	29	44	63	75	454
⋮	48	37	54	72	73	39	38	14	375

图3 超级块有效数据表示例

传统的贪心算法挑选有效数据块最小的超级块作为源数据块,它在超级块层面是最优的,但是从整个系统层面上,即在所有物理块范围内,它并不是最优的。它做垃圾回收时,没有把有效数据最小的物理块挑选出来作为源数据块。为减小固态硬盘写放大,提升系统寿命和满盘下的写性能,本文提出一种算法:在做垃圾回收时,挑选每个 die 上有效数据最小的源物理块,通过重组超级块,使重组后的超级块上面有效数据量最小,然后把该超级块作为源超级块来进行垃圾回收,减小了写放大,改善了闪存的寿命和性能。

2 算法介绍

2.1 重组超级块

传统超级块是每个 die 上具有同样块编号的物理块组成的。在保证超级块并行度的前提下,为减小写放大,新算法打破了这个约束,它让不同 die 上具有不同编号的物理块组成超级块。具体来说,就是在挑选源超级块的时候,新算法把每个 die 上有效数据量最小的物理块组成一个新的超级块,如图4所示。

	Die 0		Die 1		Die 2		Die 3		有效数据个数
	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	
Block 0	40	41	99	97	42	12	4	52	387
Block 1	49	23	93	4	48	85	0	83	385
Block 2	40	56	98	42	52	7	47	53	395
Block 3	95	50	65	18	68	2	12	11	321
Block 4	29	49	33	7	4	52	68	72	314
Block 5	75	10	45	14	57	24	30	11	266
Block 6	17	57	64	88	88	50	69	39	472
Block 7	98	73	91	95	15	90	71	39	572
Block 8	67	58	82	36	29	44	63	75	454
⋮	48	37	54	72	73	39	38	14	375
↓									
Block x	Block 6	Block 5	Block 4	Block 1	Block 4	Block 3	Block 1	Block 3	81

图4 重组超级块示例

新的超级块 x 由每个 die 上有效数据最小的物理块组成,有效数据为 81,和前面未重组挑选出来的超级块 5 相比,有效数据量大大减小。假设每个闪存块能写入的数据量为 100,即 TPC = 800,根据公式得出未重组超级块的写放大为

1.5, 而重组超级块后写放大减小为 1.1。

在做垃圾回收之前,通过挑选每个 die 上有效数据量最小的物理块重组源超级块,使得该超级块具有最小的有效数据,从而保证了每次垃圾回收引入的写放大最小,因此能大幅提升闪存寿命和系统性能。

垃圾回收具体算法步骤如下:

- 1) 遍历每个 die 上物理块的有效数据表,找到每个 die 上有效数据最小的物理块;
- 2) 把这些物理块组成一个新的超级块,作为垃圾回收的源超级块;
- 3) 把源超级块上面的有效数据搬移到目标超级块;
- 4) 当源超级上面的数据搬移完成后,更新超级块信息表(见 2.2.1 节);
- 5) 擦除源超级块,添加到可用超级块的队列,并分别增加该超级块里面的物理块的擦除次数(见 2.2.2 节)。

2.2 新的数据结构和表

2.2.1 超级块信息表

传统的超级块由于是由每个 die 上具有同样编号的物理块组成,超级块编号和物理块具有同样的编号。但由于新的超级块是由每个 die 上不同编号的物理块组成,因此 FTL 需要一张超级块信息表,来记录每个超级块是由哪些物理块组成。

超级块号	Die 0		Die 1		Die 2		Die 3	
	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1	plane 0	plane 1
0	Block a	Block b	Block c	Block d	Block e	Block f	Block g	Block h
1								
2								
⋮								
N-1								

图 5 超级块信息表

图 5 中, N 为每个 die 上的每个 plane 上的物理块个数, $a, b, \dots$ 在 $[0, N-1]$ 范围内。

2.2.2 基于物理块的擦除次数表

FTL 需要维护一张超级块擦除次数表,用以磨损平衡算法、读干扰处理算法等。传统的超级块由于其物理块是固定的,因此,每一个超级块的擦写次数就代表了每个物理块的擦写次数。现在新算法重组超级块,导致超级块内部的每个物理块次数并不相等,因此 FTL 不能基于超级块来统计擦除次数,而是要基于每个物理块记录擦写次数。

2.2.3 基于物理块的读次数表

FTL 有一张记录物理块的读次数的表,读干扰处理算法会在物理块读次数达到一定阈值之前,即数据变坏之前把数据写入到新的地方(数据刷新)。和擦除次数一样,传统的方式是基于超级块的读次数。同样的,由于超级块的重组,现在 FTL 需要记录每个物理块(而不是超级块)的读次数。

2.2.4 更多的内存资源需求

在实现上,FTL 需要维护超级块信息表以及每个物理块的有效数据个数、擦除次数和读取次数,和传统基于非重组超级块的算法相比,它需要更多的内存空间来存储这些 FTL 所需的数据,具体来说,所需内存空间是之前的 N 倍,其中 N 为系统中 die 的个数。但由于现在的 SSD 都配有大容量的 DRAM,系统完全能支持这些额外的资源开销。

2.3 新的读干扰算法和磨损均衡算法

2.3.1 基于物理块的刷新算法

在传统的读干扰算法中,即使是超级块中的某一个物理块读次数过多,它也刷新整个超级块(即超级块中所有物理块都需要重新读出来,然后写到一个新的超级块)。这种处理方式对闪存寿命和性能都有影响。现在 FTL 有了基于物理块的读次数表,FTL 可以做基于物理块的刷新,即 FTL 只需要刷新超过读阈值的那个物理块,超级块中其他未超过读次数阈值的物理块则无需刷新。

超级块x	Block A	Block B	Block C	Block D	Block E	Block F	Block G	Block H
空闲超级块	block a	block b	block c	block d	block e	block f	block g	block h

图 6 基于物理块的刷新示例

在完成刷新后(如图 6,物理块上的数据从 Block C 搬到 block c),超级块 x 的信息表也需要更新如图 7。

超级块x	Block A	Block B	Block C	Block D	Block E	Block F	Block G	Block H
超级块x	block A	block B	block c	block D	block E	block F	block G	block H

图 7 刷新操作完成后更新超级块信息

基于物理块的刷新算法能大幅减少不必要的的数据搬移,从而提升系统寿命和性能。

2.3.2 基于物理块磨损均衡算法

磨损均衡算法有动态磨损均衡算法和静态磨损均衡算法^[4],传统 FTL 都是基于超级块来实现磨损均衡。

但由于现在每个超级块中的物理块擦写次数都不一样,因此 FTL 不能再基于超级块来做磨损均衡,相应的,FTL 需要基于物理块来做磨损均衡。

FTL 在做动态磨损均衡算法的时候,基于超级块的实现方式是挑选最小擦写次数的空闲超级块作为即将要写入的超级块,而基于物理块的实现方式变成从每个 die 上挑选每个最小擦写次数的空闲物理块来组成一个超级块,然后把这个超级块作为即将要写入的超级块。

静态磨损均衡算法是让冷数据搬移到擦写次数比较多的块上面去。FTL 在做静态磨损均衡算法的时候,基于超级块的实现方式是把冷数据从擦写次数比较少的超级块搬到擦写次数比较多的超级块上面。参考文献^[4]及其提到的其他静态磨损做法都是基于超级块的。但是随着现在的闪存物理块容量越来越大,而且 SSD 容量越来越大(需要更多的 die),导致一个超级块的容量很大。因此,一个超级块可能有冷热数据并存的情况。相对于超级块,更小的单个物理块存在冷热数据并存的概率则大大减小,从而可以避免在做静态磨损均衡的时候进行热数据的搬移。现在 FTL 做基于物理块的静态磨损均衡,做法类似,即在每个 die 上,把冷数据从擦写次数比较少的块搬到擦写次数比较多的物理块上面,完成之后需要更新源超级块信息。

3 实验结果与分析

实验使用 FlashSIM^[8] 仿真软件来测试重组超级块算法对 SSD 寿命(通过写放大反应)和满盘时 SSD 写入性能。FlashSIM 是一款开源的 SSD 模拟器,它是一款事件驱动的、模块化的基于 C++ 的模拟器,内置了多种 FTL 策略,能够提供响应时间、能耗的模拟及许多额外的统计信息。本实验在

FlashSIM 软件中 DFTL^[9] 的基础上, 实现了基于传统超级块算法和重组超级块算法, 其中垃圾回收策略为贪心算法。

表 1 SSD 相关参数

参数	值
闪存 Die 的大小/GB	32
Die 的个数	4
物理块(block) 的个数	512
每个 die 上 plane 的个数	2
每个 block 上 page 的个数	768
平均写入时间/ μs	820
平均读取时间/ μs	76
通道个数	2
每个通道传输速率/($\text{Mt} \cdot \text{s}^{-1}$)	600
设备裸容量/GB(二进制)	128
用户容量/GB(十进制)	120
超级块大小/MB/	288
Over Provisioning/%	14
每个超级块可写入的数据块的个数(TPC)	24 576

测试内容:

满盘下的写放大, 顺序和随机写入性能。

表 2 传统超级块算法与生组超级块算法实验结果对比

算法	源超级块有效数据个数	写放大	256 KB 顺序写速度/($\text{MB} \cdot \text{s}^{-1}$)	4 KB 随机写速度/kIOPS
传统超级块算法	22 000	6.82	17	4
重组超级块算法	5 500	2.53	83	19

表 3 传统超级块算法与生组超级块算法内存需求对比

算法	超级块信息表/KB	擦除次数表/KB	有效数据表/KB	读次数表/KB
传统超级块算法	0	1	1	2
重组超级块算法	4	4	4	8

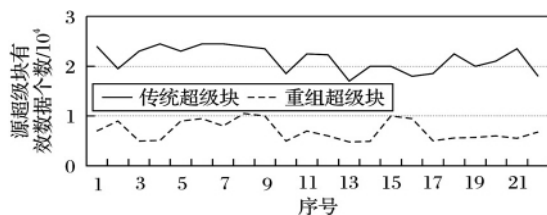


图 8 源数据块上有效数据历史

4 结语

当 SSD 的 OP(Over Provisioning) 比较小的时候, 基于超级块的垃圾回收会引入很大的写放大, SSD 寿命短、性能差。本文通过超级块重组的方式, 构建出一个有效数据最小的超级块作为垃圾回收的源超级块, 减小了垃圾回收导致的内部数据搬移, 提升了 SSD 寿命和满盘性能。由于新的超级块来自 die 上不同的物理块, 因此 FTL 的磨损均衡算法和读干扰处理算法要改成基于物理块的处理方式, 其代价是需要更多的片上 RAM 资源, 但由于 SSD 一般都配有大容量 DRAM, 所以额外的资源需求都能得到满足。

参考文献:

- [1] SSDFans. 深入浅出 SSD[M]. 北京: 机械工业出版社, 2018: 122 - 152.
- [2] 杜晨杰. 基于 NAND 闪存的固态硬盘关键技术研究[D]. 杭州: 杭州电子科技大学, 2015: 12 - 12.
- [3] D' ABREU M A. NAND flash memory — Product trends, technolo-

实验步骤:

- 1) 用 4 KB 随机写入填满整盘;
- 2) 256 KB 大小顺序写入, 总共写入 10 GB 数据, 测试顺序写入性能和写放大;
- 3) 重新按步骤 1) 用 4 KB 随机写入填满整盘;
- 4) 4 KB 大小随机写入, 总共写入 10 GB 数据, 测试随机写入性能和写放大。

用以上步骤, 分别测试基于传统超级块和重组超级块算法下的性能和写放大, 记录每次作垃圾回收挑选出来的源超级块上面有效数据个数, 结果如图 8、表 2 所示。

实验结果表明, 通过重组超级块, 源超级块上面的有效数据个数大大减少, 意味着回收一个物理块, 内部需要搬移的数据大大减小, 从而写放大变小, 最终反映出 SSD 性能和寿命得到明显的提升。

在内存资源消耗上, 新的算法要求更多的内存, 具体差异如表 3。

在 4 个 die 的配置上, 新的算法比传统算法额外需要 16 KB 的 RAM 资源, 这个数值随着 die 的个数增多呈线性增加。但现在 SSD 很多都有几百 MB 甚至上 GB(取决于 SSD 容量大小) 的 DRAM, 所以额外的内存需求应该不是问题。

- gy overview, and technical challenges [C]// Proceedings of the 2011 Asian Test Symposium. Washington, DC: IEEE Computer Society, 2011: 463 - 463.
- [4] KANG J U, JO H, KIM J-S, et al. A superblock-based flash translation layer for NAND flash memory [C]// Proceedings of the 6th ACM & IEEE International Conference on Embedded software. New York: ACM, 2006: 161 - 170.
- [5] 王晋阳, 严华. 基于逻辑页冷热分离的 NAND 闪存磨损均衡算法[J]. 计算机应用, 2016, 36(5): 1430 - 1433.
- [6] HU X Y, ELEFTHERIOU E, HAAS R, et al. Write amplification analysis in flash-based solid state drives [C]// Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference. New York: ACM, 2009: Article No. 10.
- [7] 赵峰, 刘博妍, 朱戈. 关于 NAND 闪存损耗均衡算法的优化[J]. 南华大学学报(自然科学版), 2017, 31(3): 86 - 91.
- [8] KIM Y, TAURAS B, GUPTA A, et al. FlashSIM: A simulator for NAND flash-based solid-state drives [C]// Proceedings of the 2009 First International Conference on Advances in System Simulation. Washington DC: IEEE Computer Society, 2009: 125 - 131.
- [9] GUPTA A, KIM Y, URGANONKAR B. DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings [C]// Proceedings of the 14th International Conference on Architecture Support for Programming Languages and Operating Systems. New York: ACM, 2009: 229 - 240.